

JARVIS

A Controlled Experimental Framework for Error-Driven Continual Learning in Small From-Scratch Language Models

Aditya Bhatia

Independent Research · Technical Research Manuscript · September 2026

Scope of manuscript

Archived Generation-10 observations, system design, and a preregisterable V7 experimental protocol

Research integrity statement

The manuscript separates measured observations from proposed experiments. It makes no claim of frontier-model parity or general intelligence.

Abstract

JARVIS is a from-scratch research system designed to investigate continual learning as a closed-loop experimental problem rather than as a single optimization run. The system combines a compact transformer-style language model with persistent checkpoint lineage, public-web information acquisition, retrieval, deterministic tools, benchmarking, and discrete architecture search. The archived V6 trajectory records successive accepted modifications from an approximately 13-million-parameter regime to 156,144,084 tracked parameters by Generation 10, alongside 3,520 recorded training steps, 4,643,495 corpus characters, 40 ingested web pages, 103 persistent memory records, and nine accepted architecture changes. These quantities describe the state of the experimental platform; they do not establish broad language competence. In particular, qualitative testing exposed a separation between optimization and user-visible capability: low recorded loss could coexist with poor arithmetic behavior, retrieval could surface irrelevant evidence, and larger candidate configurations could exceed the available GPU memory budget. The next research phase therefore treats capability failures as explicit intervention targets. JARVIS V7 is specified as a controlled loop in which the system evaluates a fixed benchmark, diagnoses each failure across knowledge, retrieval, reasoning, tool-routing, optimization, and infrastructure causes, acquires relevant permitted public information when justified, performs a bounded learning intervention, and retains the candidate only when target improvement, regression control, provenance, and resource criteria are satisfied. The study defines operational distinctions between information access and parameterized learning and proposes matched ablations for retrieval, tools, replay, generic continuation, and architecture evolution. The primary contribution is methodological: an inspectable, resource-aware framework for measuring whether a small neural system can improve by choosing what to learn next, while preserving enough state and provenance to make both positive and negative results scientifically interpretable.

Keywords. continual learning · error-driven learning · small language models · retrieval · information acquisition · architecture evolution · evaluation · low-resource AI · reproducibility

1. Introduction

Contemporary language-model development is usually organized as a forward pipeline: select an architecture, assemble training data, optimize parameters, evaluate a checkpoint, and deploy. That pipeline is appropriate when the main experimental question is how performance scales with data, compute, and parameter count. It is less informative when the research question concerns the learning process itself: how the system detects failure, how it determines what information is missing, how it selects an intervention, and how it decides whether a change was genuinely beneficial.

JARVIS was constructed to make those questions experimentally visible. Its neural core is intentionally small enough to retrain repeatedly on accessible hardware. Around that core sits an explicit control layer that records checkpoint lineage, retrieves evidence, ingests public material, invokes deterministic tools, evaluates outputs, and proposes discrete architecture changes. The surrounding infrastructure is therefore part of the experimental object. A

useful result is not merely a lower loss; it is a reproducible change in behavior whose causal pathway can be examined.

The first trajectory produced a deliberately mixed result. The engineering mechanism was viable: the system preserved successive generations and accepted nine architecture changes, reaching 156,144,084 tracked parameters by Generation 10. At the same time, the capability evidence did not grow in lockstep with capacity. The user-facing system could retrieve a relevant page while also exposing irrelevant learned chunks, and natural-language arithmetic could be mishandled even when arithmetic examples existed in the corpus. More aggressive architecture candidates also encountered GPU-memory limits. These observations motivate a stronger experimental separation among model capacity, information access, tool routing, and parameterized learning.

The central question is therefore not whether JARVIS can become large. It is whether a comparatively small model can be embedded in a persistent, measurable cycle of diagnosis and intervention. Under this framing, architecture evolution becomes one possible action among several. The controller may choose more data, better retrieval, a deterministic tool, a training intervention, a replay strategy, or an architecture mutation, but the choice should be driven by measured evidence rather than by parameter count alone.

The manuscript has two goals. First, it records the current state of the system with enough detail to define a baseline. Second, it establishes a rigorous V7 protocol that can distinguish genuine parameterized learning from answer-time retrieval, tool use, or memorization. The resulting research program is intended to remain informative even when the main hypothesis is not supported.

2. Research Positioning and Related Work

Continual learning studies adaptation to evolving data or tasks while controlling forgetting. Surveys focused on language models describe continual pretraining, domain adaptation, and continual fine-tuning as major regimes and emphasize the evaluation difficulties created when previous capabilities degrade after new updates [1,2]. The JARVIS setting is narrower: the model is trained from scratch at small scale, and the controller itself selects the next intervention based on observed failures.

Retrieval-augmented generation (RAG) separates knowledge access from parametric storage by conditioning generation on retrieved external evidence [3]. Recent surveys emphasize that retrieval quality, grounding, source selection, and robustness are first-order system variables rather than implementation details [4]. JARVIS adopts this distinction explicitly. A retrieval-only improvement is recorded as an access-layer effect unless controlled experiments demonstrate subsequent parameter-only transfer.

Tool-augmented language modeling further separates generation from deterministic computation. Toolformer and ReAct demonstrate that language models can be paired with external operations or information sources to extend their effective capabilities [5,6]. The JARVIS contribution is not to introduce a new tool-use primitive; it is to make tool selection a measurable subsystem whose effects can be separated from neural learning.

Self-refinement and self-play methods show that iterative feedback or synthetic data can improve model behavior under some conditions [7,8]. These studies motivate the possibility of closed-loop improvement but also raise a methodological problem: the source of improvement can be difficult to attribute when feedback, retrieval, data generation, and parameter updates occur together. JARVIS addresses this problem by predefining intervention classes and matched ablations.

Finally, compute and data constraints matter. Compute-optimal scaling and data-constrained model studies demonstrate that parameter growth cannot be interpreted independently of training data and compute [9,10]. In a resource-limited independent-research setting, the practical objective is therefore closer to information gained per unit compute than to unrestricted scaling.

3. Research Gap and Scientific Contribution

The JARVIS research gap is a systems-level one. Existing work separately studies continual learning, retrieval, tools, self-improvement, and scaling. JARVIS asks how these mechanisms should interact when a controller must decide which mechanism to change after observing a failure. The key experimental challenge is attribution. A system can appear to “learn” while the actual improvement came from retrieval. It can appear to reason better because a deterministic tool was added. It can lower loss because it overfit the training corpus. The experiment must therefore identify the route to success, not only the final answer.

The first contribution is an operational definition of parameterized learning. JARVIS treats a learning event as evidence of model improvement only when a candidate checkpoint improves on unseen target items under controlled evaluation, with external retrieval and deterministic tools either disabled or explicitly matched, and when prior validated capabilities remain within the regression tolerance. This definition is intentionally conservative. It prevents a system from receiving learning credit for simply finding an answer elsewhere.

The second contribution is a failure-to-intervention framework. Each failure is classified across a finite taxonomy and mapped to candidate actions. The controller is therefore an experimental policy over interventions, not a generic “self-improvement” prompt. The third contribution is resource-aware architecture evolution: candidate architectures are judged by feasibility and held-out improvement, while out-of-memory events remain archived as observations about the feasible search region.

The fourth contribution is experimental governance. JARVIS keeps source code, checkpoints, corpus versions, benchmark versions, acquisition provenance, and intervention decisions linked through immutable identifiers. The objective is not merely to make the project reproducible; it is to make every accepted change falsifiable and auditable.

4. Research Questions and Hypotheses

The primary question is: Can a relatively small, from-scratch language-model system identify capability failures, acquire relevant permitted public information, learn from targeted material, evaluate improvement on unseen tasks, and retain beneficial changes over repeated iterations?

- H1 — Error-driven learning: at approximately fixed model size and matched compute, targeted learning from diagnosed capability gaps will yield larger and more reliable held-out gains than generic continuation.
- H2 — Information access: answer-time retrieval can improve factual performance without changing model parameters; such gains must be credited to information access unless parameter-only transfer is demonstrated.
- H3 — Retention: an intervention is successful only when its target capability improves while previously validated capabilities remain within a predefined regression tolerance.
- H4 — Bounded evolution: architecture mutations should be accepted only when resource-feasible and beneficial on held-out evaluation beyond matched continuation.
- H5 — Evaluation: versioned, unseen benchmarks and regression suites provide a more trustworthy longitudinal signal than raw training loss when the data distribution changes.

These hypotheses are intentionally separable. A result can support H2 while failing H1; it can support H1 on one capability while exposing regression elsewhere; or it can show that architecture evolution adds little value after the learning controller is improved. This structure prevents one favorable metric from being treated as evidence for the whole system.

5. Conceptual Model: Capability as a Composite System Property

The user-visible answer is modeled as a function of interacting components. Let x be a task, M the neural state, R the retrieval subsystem, T the deterministic-tool subsystem, and C the learning controller. The observed response is represented abstractly as:

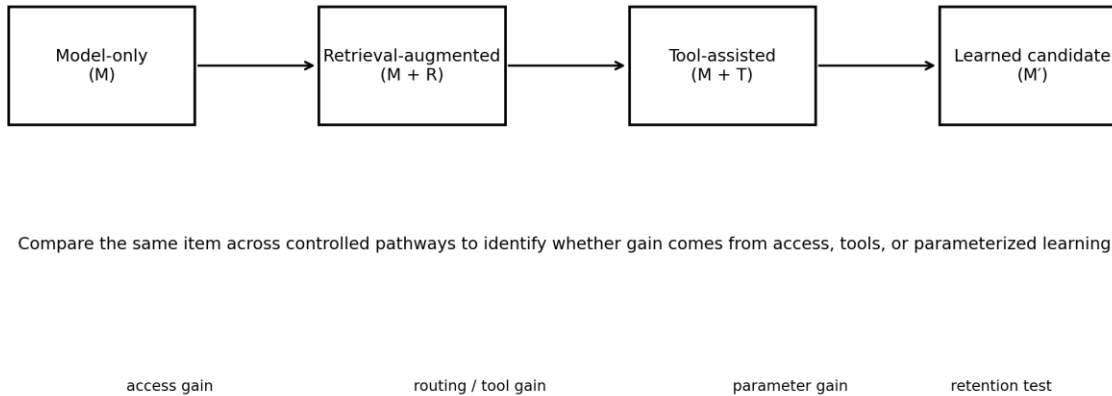
$$y = F(x; M, R, T, C).$$

An intervention may alter any one of these terms. Retrieval can change the evidence available at inference time without changing M . Tool use can change the computational route without changing M . Training changes M . Architecture search can change the representational and computational envelope within which all other components operate. The same visible answer can therefore arise through different causal pathways.

For experimental accounting, the total gain can be decomposed conceptually as:

$$G_{\text{total}} = G_{\text{access}} + G_{\text{param}} + G_{\text{route}} + G_{\text{tool}} + \epsilon.$$

The terms are not assumed to be perfectly identifiable in a single observation. Instead, the decomposition motivates controlled comparisons. The same benchmark item can be run through model-only, retrieval-augmented, tool-assisted, and candidate-model pathways. Performance changes across those pathways provide evidence about the subsystem responsible for the gain.

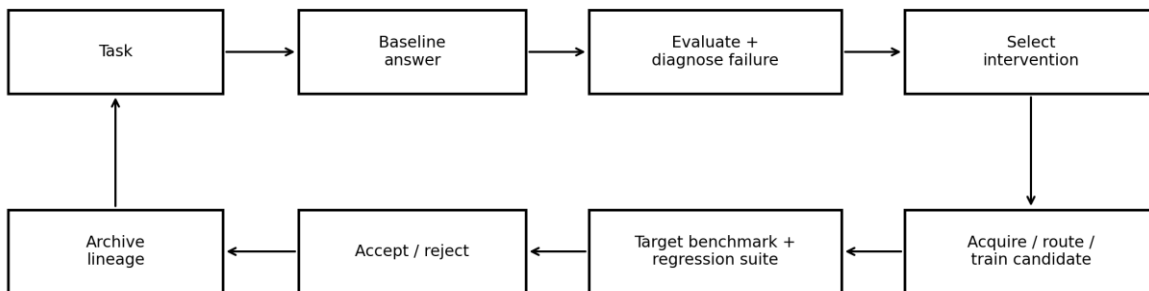


Compare the same item across controlled pathways to identify whether gain comes from access, tools, or parameterized learning.

Figure 1. Causal-attribution view of a JARVIS task. The same item is evaluated through controlled pathways so that access, tool, routing, and parameterized-learning effects are not conflated.

6. System Architecture

JARVIS is organized as a set of explicit interfaces: neural generation, persistent state, information acquisition, retrieval, deterministic tools, benchmarking, failure diagnosis, and architecture evolution. The controller sits above these mechanisms and selects interventions using the current empirical history. The architecture is modular because modularity is required for scientific attribution.



Accepted candidates become the parent state; rejected and interrupted candidates remain archived for analysis.

Figure 2. JARVIS V7 experimental control loop. The system evaluates a task, diagnoses a failure, selects an intervention, evaluates the candidate, and retains or rejects the change according to fixed criteria.

6.1 Neural core

The archived Generation-10 configuration records a 256-symbol vocabulary, $d_{\text{model}} = 384$, eight attention heads, 12 layers, seven experts, top-k routing of four, dropout = 0.0, and block size 1,280. The tracked parameter count is

156,144,084. The neural core is intentionally modest; the experiment is concerned with learning dynamics and system control rather than competitive frontier scale.

6.2 Persistent state and lineage

Every generation is associated with a configuration and checkpoint. Candidate actions and acceptance decisions are recorded in machine-readable form. Rejected candidates remain preserved because they contain information about resource limits, optimization sensitivity, and failure modes.

6.3 Acquisition layer

The acquisition layer collects permitted public information in a versioned manner. It records source URL, timestamp, query, content hash, and filtering decision. Acquisition is distinct from answer-time retrieval so that data can be traced into later learning interventions.

6.4 Retrieval layer

The retrieval layer searches a local corpus and, in the current design, can call public search services for answer-time evidence. Retrieval is evaluated for relevance, source diversity, and duplication. The research record can retain detailed evidence even when the user-facing interface displays only the final answer.

6.5 Tool layer

Deterministic tools handle exact operations for which sampling is an inappropriate mechanism, such as arithmetic or verifiable code execution. Tool routing is scored separately from answer correctness.

6.6 Evaluation and evolution layer

Evaluation produces benchmark scores, regression outcomes, resource measurements, and an intervention record. Architecture evolution is a constrained search process over candidate configurations, not an unconditional objective to increase parameters.

7. Generation-10 Baseline

Generation 10 is the parent state for the proposed V7 study. The archived record provides a concrete baseline for capacity, optimization history, information accumulation, and lineage. These quantities should be treated as descriptors of one experimental trajectory, not as normalized measures that are directly comparable with unrelated models.

Metric	Archived value	Interpretation
Generation	10	Archived state identifier
Tracked parameters	156,144,084	Model capacity
Training steps	3,520	Recorded optimization budget
Best evaluation loss	0.1163263097	Optimization metric under the archived configuration
Corpus characters	4,643,495	Indexed corpus size; characters, not tokens
Web pages	40	Recorded web ingestion
Memory records	103	Persistent state records
Accepted changes	9	Accepted architecture interventions
Device	CUDA / T4-class	Experimental resource envelope

The lineage shows that the largest tracked-capacity jump occurred at the expert-addition mutation. The tracked count increased from 134,864,328 to 156,144,084, approximately 15.8 percent. Later accepted mutations altered context or routing-related configuration without changing the tracked parameter count. This makes the trajectory useful for testing the hypothesis that capacity changes and learning-policy changes have distinct effects.

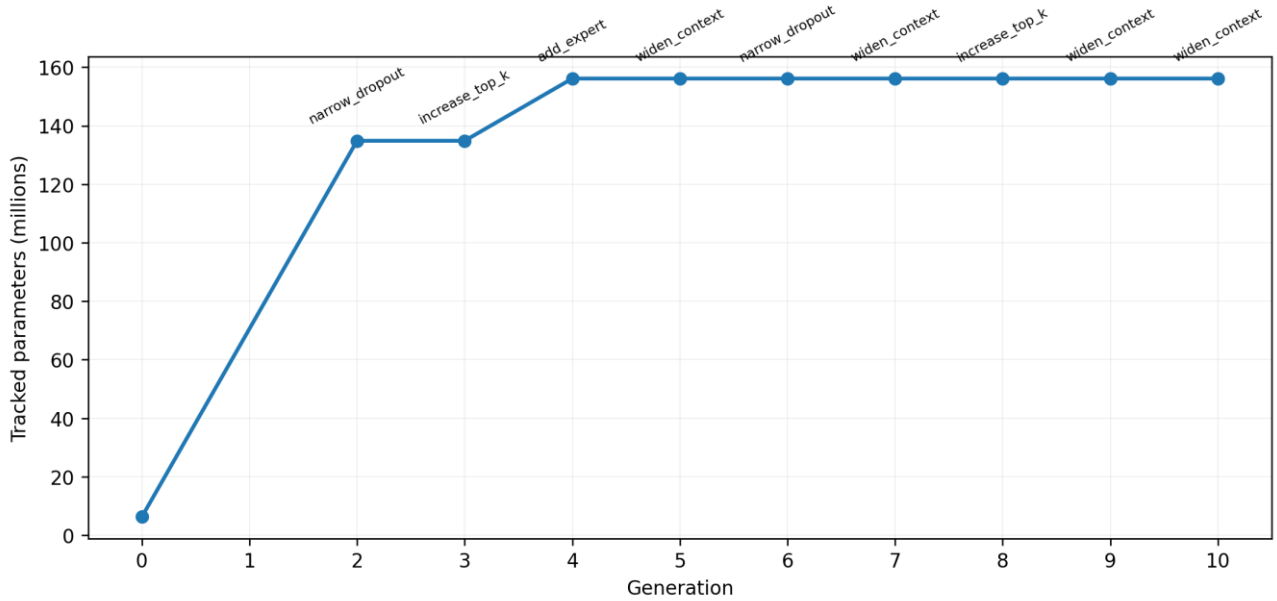


Figure 3. Archived tracked-parameter trajectory through Generation 10. The capacity jump occurs at the expert-addition mutation; later accepted changes preserve the tracked count.

Generation	Accepted action	Tracked parameters
2	narrow_dropout	134,864,328
3	increase_top_k	134,864,328
4	add_expert	156,144,084
5	widen_context	156,144,084
6	narrow_dropout	156,144,084
7	widen_context	156,144,084
8	increase_top_k	156,144,084
9	widen_context	156,144,084
10	widen_context	156,144,084

8. Experimental Instrumentation and State

A continual-learning experiment is only as strong as its instrumentation. JARVIS therefore treats state preservation as part of the scientific method. A run is not adequately described by a final checkpoint alone; the controller must preserve the decision history that produced it.

8.1 Checkpoint semantics

Each checkpoint is treated as a parentable artifact. The experiment should record a cryptographic hash, architecture configuration, optimizer configuration, training-step count, corpus version, benchmark version, and parent identifier. If a trial is interrupted, the last valid checkpoint remains a first-class artifact rather than being silently overwritten.

8.2 Controller semantics

The controller has a version separate from the model generation number. This prevents a change in policy logic from being mistaken for a model improvement. Controller versions should specify action probabilities or policies, thresholds, search procedures, and the set of allowed interventions.

8.3 Web provenance

Every acquisition event should be represented as a provenance tuple: query, timestamp, source URL, source domain, extracted-content hash, filtering result, and downstream training-task identifier. This makes it possible to trace a later capability improvement back to the public information that was used to induce it.

9. Failure Taxonomy and Diagnostic Inference

The controller needs a finite vocabulary of failure modes because learning is a conditional intervention. The taxonomy below is not asserted to be complete; it is designed to cover the main bottlenecks observed in the current system and the most common failure routes in a small language-model stack.

Failure class	Observable signal	Candidate intervention
Knowledge gap	Relevant fact absent from model-only context	Acquire targeted public information; build bounded learning set
Retrieval failure	Search returns irrelevant, duplicate, or weak evidence	Query rewrite; source ranking; filtering; diversity constraint
Reasoning failure	Evidence is present but transformation is incorrect	Targeted worked examples; verifier; decomposition
Tool-routing failure	Exact tool exists but generator answers directly	Routing classifier / rule system / tool-use training
Optimization failure	Unstable loss, poor convergence, seed sensitivity	Optimizer / schedule / batch intervention
Forgetting	Target improves while regression suite declines	Replay; regularization; lower update intensity
Infrastructure failure	OOM, interruption, corrupted artifact	Resource-aware candidate; checkpoint validation; recovery

The taxonomy supports probabilistic diagnosis. The controller can assign a distribution over causes rather than committing to a single explanation. The subsequent intervention should therefore be logged together with the diagnosis confidence. A future benchmark can then measure diagnostic accuracy independently of the learning outcome.

10. Public-Web Information Acquisition

Public-web acquisition is useful only when the information source is treated as an experimental variable. JARVIS distinguishes two information paths: batch acquisition for later training and answer-time retrieval for immediate responses. The first changes the corpus available to the optimizer; the second changes the evidence presented to the generator.

10.1 Source selection

A quality-aware acquisition system should rank sources using evidence characteristics rather than search position alone. Government and academic domains, primary research papers, technical documentation, and established reporting can receive stronger priors when relevant. Lower-quality or duplicate sources should be down-ranked. The objective is not to forbid all secondary sources but to minimize the probability that a single weak page dominates the answer.

10.2 Retrieval verification

The system should measure whether retrieved documents actually address the query. For benchmark items with known relevant sources, top-k retrieval recall and source-domain diversity can be computed. A sampled audit can estimate relevance precision and identify systematic search failures.

10.3 Search is not learning

A central scientific rule is that retrieval correctness is not parameterized learning. If the system answers a question because a web source supplies the fact, the event should be stored as an information-access success. Only a later parameter-only test can establish that the model internalized the information to a measurable degree.

11. Deterministic Tools and Interface Routing

Exact computation is a natural example of a task that should not be entrusted to unconstrained language generation. For an expression such as $55 + 77$, the system should route the operation to a calculator and return 132. The language model need not approximate an exact symbolic function from a probability distribution.

Tool routing should be evaluated separately. A route can be wrong even when the eventual answer is correct, and a route can be correct even if the downstream tool execution fails. The log should therefore contain task type, chosen route, tool execution result, and final answer. This decomposition makes it possible to compare neural competence with system competence.

The same logic applies to code execution, structured transformations, unit conversion, and other verifiable operations. The experimental question is not whether every capability should be moved into tools. It is whether a controller can learn to choose tools when they offer a measurable reliability advantage.

12. Error-Driven Continual-Learning Algorithm

The V7 intervention begins from a frozen parent checkpoint M . The model is evaluated on the target benchmark B_{target} and the regression suite G . Failures are classified, an intervention is selected, and a candidate M' is produced under a fixed resource budget. The candidate is then tested on the same target and regression sets.

$\text{Accept}(M') = 1$ iff $\Delta_{\text{target}} \geq \delta$ and $\Delta_{\text{regression}} \geq -\tau$ and $\text{feasible}(M') = 1$ and $\text{provenance}(M') = 1$.

Here Δ_{target} is the change in target performance, δ is the minimum practically meaningful gain, $\Delta_{\text{regression}}$ is the change in previously validated performance, τ is the maximum tolerated decline, feasible denotes that the candidate satisfies the resource envelope, and provenance denotes that the state and data required to reconstruct the intervention are complete.

The acceptance rule deliberately separates improvement from feasibility. An intervention that improves the target capability but requires an unavailable amount of GPU memory is not a successful candidate. Likewise, an intervention that improves a narrow target while causing large regressions should not become the new parent state.

12.1 Replay and retention

Replay is included as a retention mechanism. The replay buffer should contain examples representing previously validated capabilities and should be versioned with the experiment. The ratio of new target data to replay data becomes an experimental variable. This permits an ablation between unconstrained targeted learning and targeted learning with explicit retention pressure.

12.2 Intervention budget

Intervention budgets should be expressed in steps, wall-clock time, memory, or another reproducible resource unit. Matched budgets are essential for comparing targeted and generic continuation. The controller should not receive hidden extra compute simply because an intervention is more complex to implement.

13. Benchmark Engineering

Benchmark design is the main defense against self-confirming learning loops. The system should maintain separate development, validation, regression, and final-test partitions. Development items support controller design. Validation items support intervention selection. The regression suite protects previously validated capabilities. The final test set remains untouched until the evaluation block is complete.

Category	Suggested item count	Primary grading rule
Arithmetic	100–200	Exact numeric correctness
Science concepts	100–200	Fact correctness + explanation rubric

General knowledge	100–200	Correctness + abstention when evidence is insufficient
Reading comprehension	100–200	Answer supported by passage
Code reasoning	100–200	Execution / exact output where feasible
Current-information QA	100–200	Evidence-grounded correctness
Tool routing	100–200	Correct tool / no-tool decision

The important property is independence, not the exact counts. Training material should not be copied into evaluation. When web sources are used to generate training material, the evaluation set should preferably rely on different source instances or surface forms. Repeated templates should also be avoided because a model can obtain an apparently large score by memorizing superficial patterns.

14. Evaluation Metrics and Statistical Plan

The primary outcome is held-out capability change. For categorical tasks, accuracy and confidence intervals are straightforward. Because the same benchmark items can be evaluated before and after an intervention, paired analyses are preferable to independent-sample assumptions.

A paired bootstrap can estimate confidence intervals for the change in per-item correctness. McNemar-type paired tests may be appropriate for binary outcomes on sufficiently large matched samples. For small intervention batches, permutation or exact methods can be used. Effect sizes should accompany significance tests so that a statistically detectable change is not mistaken for a practically useful improvement.

Secondary metrics include regression rate, retrieval relevance, source diversity, routing accuracy, tool success, latency, memory use, and compute. Training loss remains an optimization diagnostic. It should not be treated as a universal capability score, especially when the training corpus changes between interventions.

14.1 Scorer validation

Automated scoring can produce false positives and false negatives. A sampled manual audit should therefore be built into the evaluation plan. The scorer itself should be versioned, and any change in scoring logic should trigger re-evaluation of stored outputs when feasible. This is particularly important when the research question concerns self-evaluation or controller reliability.

15. Archived V6 Findings

The archived trajectory supports three observations. First, the infrastructure works sufficiently to preserve an evolving experiment. Second, optimization progresses measurably under the archived configuration, as reflected in the recorded 3,520 training steps and best evaluation loss of approximately 0.1163. Third, user-visible capability is not explained by those optimization metrics alone.

The distinction is visible in the chat behavior. The system can return useful web evidence for a factual question, yet the surrounding interface may also surface irrelevant retrieved fragments. A simple arithmetic question can trigger a learned retrieval path rather than an exact calculator. These are different failure classes even if both appear to the user as “the model got it wrong.”

The architecture lineage also illustrates the danger of treating parameter count as a quality metric. Several later accepted changes preserved the same 156,144,084 tracked parameter count. The system thus has already produced multiple internal configurations at the same capacity, creating an opportunity to test whether learning policy and routing matter independently of size.

16. Failure Analysis as Causal Attribution

The most informative V6 lesson is not a single benchmark score; it is the gap between what the system can access and what the model can internalize. This motivates a controlled attribution sequence. First evaluate model-only. Then allow retrieval without parameter updates. Then evaluate a parameter-only candidate without retrieval. Finally evaluate the integrated system. The differences among these conditions form a causal map of where the gain originates.

A second attribution problem concerns generic continuation. If a targeted intervention uses 100 steps and a generic baseline uses 100 steps, the comparison is interpretable. If the targeted intervention also receives more data, more retrieval calls, and more tuning attempts, the result becomes difficult to attribute. V7 therefore treats compute and information budgets as explicit controls.

A third attribution problem concerns repeated evaluation. If the controller repeatedly checks the same validation items and chooses only changes that improve them, it can overfit the validation set. The untouched final test partition is therefore not optional. It is necessary to establish that a candidate generalized beyond the controller’s selection pressure.

17. V7 Controlled Experimental Program

V7 begins from the archived Generation-10 state and evaluates a set of intervention families under matched budgets. The controller is permitted to select among these families only after the parent state has been evaluated and failures have been diagnosed.

Condition	Intervention	Purpose
A	Generic continued training	Baseline for ordinary continuation
B	Retrieval only	Measure answer-time access without parameter change
C	Targeted learning, no replay	Measure adaptation without explicit retention control
D	Targeted learning + replay	Measure targeted learning with retention
E	Targeted learning + retrieval + tools	Measure integrated system behavior
F	Bounded architecture mutation after E	Test whether capacity adds value after the learning controller is improved

Each condition should be repeated across multiple random seeds when feasible. The parent checkpoint must remain untouched. A candidate that fails acceptance is not applied to the parent, but its output and logs remain archived. This design makes rejected candidates part of the evidence rather than noise.

18. Ablations and Confound Control

Ablations are designed to isolate four mechanisms: information access, parameterized learning, retention, and architecture. At minimum, V7 should compare retrieval-only with parameter-only, targeted learning with and without replay, fixed architecture with bounded search, and integrated routing with explicit tool disabling on selected neural-competence benchmarks.

Variable	Controlled comparison	Question answered
Retrieval	M vs M+R	Does external evidence explain the gain?
Parameter update	M+R vs M' without R	Did the model internalize the information?
Replay	targeted vs targeted+replay	Does retention control reduce forgetting?
Architecture	fixed vs bounded mutation	Does more capacity add held-out value?
Generic training	generic continuation vs targeted	Is targeting itself useful?
Source quality	high-prior vs mixed-source acquisition	Does provenance improve answer reliability?

Confounds should be reviewed before every experimental block. Corpus size, training steps, seed, optimizer, hardware, retrieval budget, and tool availability should be recorded. The controller code must also be versioned separately from the candidate model.

19. Resource-Aware Architecture Search

The V6 trajectory demonstrates why resource constraints belong inside the research design. Larger candidate configurations encountered CUDA out-of-memory failures on the available T4-class accelerator. Such events define the feasible frontier of the search space and can be used to construct a resource model.

A candidate should therefore be represented not only by parameter count but also by memory footprint, step time, context length, and checkpoint size. One possible selection objective is improvement per unit compute:

$$\text{Utility}(M') = \Delta_{\text{capability}} / (\text{compute_cost} + \lambda \cdot \text{memory_cost}).$$

The scalar utility is illustrative rather than mandatory. The key requirement is that resource use be recorded and that architecture changes be judged against a fixed computational budget. A candidate that is slightly better but twice as expensive may be scientifically less useful in a constrained setting than a candidate with a smaller gain and dramatically lower cost.

20. Reproducibility and Experimental Governance

A reproducible JARVIS run requires more than source code. The complete state consists of model checkpoint, model configuration, controller version, source commit, data version, benchmark version, seed, optimizer configuration, hardware description, acquisition provenance, and evaluation outputs.

- Freeze benchmark partitions before the intervention block begins.
- Hash every checkpoint and major data artifact.
- Record all acquisition queries, URLs, timestamps, and filtering decisions.
- Store raw outputs for every benchmark item.
- Run the regression suite after each accepted update.
- Preserve rejected candidates and interrupted runs.
- Keep an untouched copy of the parent checkpoint for replication.
- Separate user-facing output from internal research provenance.

20.1 Recovery and persistence

Because notebook runtimes are ephemeral, checkpoint recovery is a scientific requirement as well as an operational one. A release artifact or equivalent persistent store should contain enough state to reconstruct the parent model and its metadata. Future runs should automatically verify the artifact checksum before restoring it. The baseline should never be overwritten by a candidate update.

21. Threats to Validity

Internal validity can be threatened by data overlap, scorer errors, unequal compute, and adaptive reuse of validation items. The protocol addresses these with versioned datasets, fixed budgets, scorer auditing, and an untouched final test set.

Construct validity is also important. “Learning” can be overstated if retrieval, tools, or memorization account for the observed behavior. The explicit parameter-only transfer condition is intended to strengthen the construct of parameterized learning.

External validity is limited by scale and hardware. The system is not trained at contemporary frontier scale, and its conclusions should not be generalized to large commercial models without further evidence. The intended contribution is methodological evidence about small, resource-constrained continual-learning systems.

Ecological validity is constrained by the benchmark. Human users ask noisy, multi-step, and ambiguous questions, whereas fixed benchmarks are cleaner. A later phase should therefore supplement exact tests with realistic interaction traces while preserving an untouched evaluation partition.

22. Falsification and Outcome Interpretation

Observed result	What it would support	What it would not establish
Targeted learning improves target + low regression	H1 under the tested conditions	Open-ended self-improvement
Retrieval-only improves facts	H2 / access benefit	Parameterized learning
Target improves + strong regression	Learning with poor retention	Successful continual learning

Architecture grows + no held-out gain	Weak evidence for capacity as useful intervention	That larger models are universally unhelpful
No gain despite good acquisition/routing	Possible optimization or representation bottleneck	That the continual-learning idea is impossible
Repeated controller failures	Need for better diagnosis/evaluation	Failure of the broader research question

The controller should be evaluated as a scientific object in its own right. If it consistently selects the wrong intervention, the result is not merely a bad model; it is evidence that automated diagnosis is inadequate. This is valuable because it narrows the next experimental question.

23. Discussion

JARVIS began as an architecture-evolution exercise, but the archived trajectory suggests a more interesting scientific problem. Parameter count is easy to observe, whereas the ability to turn information into durable, transferable capability is harder to measure. The project therefore shifts from “how large can the model become?” to “how efficiently can the system determine what it should change next?”

This shift also changes the role of the model. The neural core becomes one component in a learning system. Retrieval, tools, memory, evaluation, and architecture are not accessories; they are experimental variables. The resulting framework is closer to a small laboratory platform than to a conventional one-shot language-model training pipeline.

For an independent researcher, this framing is practical. Compute constraints cannot be removed, but they can be measured. Failed candidates can become evidence about the feasible design space. Repeated small experiments can yield a richer causal record than a single large training run. The result is a research strategy in which every compute cycle is expected to answer a specific question.

24. Limitations and Non-Claims

The current evidence has substantial limitations. The model is small, the documented corpus is modest relative to contemporary language-model training, and the archived benchmark regime is still under development. The current results therefore do not justify claims of broad language understanding, human-level reasoning, or frontier-model competitiveness.

The V7 architecture is a proposal, not an observed result. Until the controlled experiments are executed, the effect of error-driven learning, replay, retrieval quality, and architecture evolution remains a hypothesis. The paper intentionally avoids converting the proposed protocol into a claimed result.

The web-access system also introduces dependence on search quality and source availability. Search engines change behavior, pages disappear, and source quality is heterogeneous. Reproducibility therefore requires storing provenance artifacts rather than assuming that repeating a query at a later date produces the same results.

25. Conclusion

JARVIS provides a concrete experimental platform for studying error-driven continual learning under constrained compute. The archived Generation-10 state demonstrates a persistent architecture-evolution and evaluation infrastructure, while the observed failure modes show why parameter count and training loss alone are insufficient measures of capability.

The proposed V7 program turns those observations into a controlled research agenda. The system first measures failure, then diagnoses it, acquires information or routes to tools as appropriate, applies a bounded learning intervention, and evaluates the result on unseen tasks while monitoring regression and resource use. Accepted changes become part of a persistent lineage; rejected changes remain available for analysis.

The scientific objective is deliberately modest and testable: determine whether a small, from-scratch model can improve more reliably when the learning policy is driven by diagnosed errors rather than by undirected continuation or architecture scaling alone. Whether the final answer is positive or negative, a controlled experiment can make the next engineering decision more evidence-based.

References

- [1] Shi, H., Xu, Z., Wang, H., et al. “Continual Learning of Large Language Models: A Comprehensive Survey.” arXiv:2404.16789, 2024.
- [2] Wu, T., Luo, L., Li, Y.-F., Pan, S., Vu, T.-T., and Haffari, G. “Continual Learning for Large Language Models: A Survey.” arXiv:2402.01364, 2024.
- [3] Lewis, P., Perez, E., Piktus, A., et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” NeurIPS, 2020. arXiv:2005.11401.
- [4] Sharma, C. “Retrieval-Augmented Generation: A Comprehensive Survey of Architectures, Enhancements, and Robustness Frontiers.” arXiv:2506.00054, 2025.
- [5] Schick, T., Dwivedi-Yu, J., Dessì, R., et al. “Toolformer: Language Models Can Teach Themselves to Use Tools.” NeurIPS, 2023. arXiv:2302.04761.
- [6] Yao, S., Zhao, J., Yu, D., et al. “ReAct: Synergizing Reasoning and Acting in Language Models.” ICLR, 2023. arXiv:2210.03629.
- [7] Madaan, A., Tandon, N., Clark, P., et al. “Self-Refine: Iterative Refinement with Self-Feedback.” NeurIPS, 2023.
- [8] Chen, Z., et al. “Self-Play Fine-Tuning Converts Weak Language Models to Strong Language Models.” arXiv:2401.01335, 2024.
- [9] Hoffmann, J., Borgeaud, S., Mensch, A., et al. “Training Compute-Optimal Large Language Models.” NeurIPS, 2022. arXiv:2203.15556.
- [10] Muennighoff, N., Rush, A., Tazi, N., et al. “Scaling Data-Constrained Language Models.” arXiv:2305.16264, 2023.
- [11] Kaplan, J., McCandlish, S., Henighan, T., et al. “Scaling Laws for Neural Language Models.” arXiv:2001.08361, 2020.
- [12] Kirkpatrick, J., Pascanu, O., Rabinowitz, N., et al. “Overcoming Catastrophic Forgetting in Neural Networks.” PNAS, 2017.
- [13] Cobbe, K., Kosaraju, J., Bavarian, M., et al. “Training Verifiers to Solve Math Word Problems.” arXiv:2110.14168, 2021.
- [14] Vaswani, A., Shazeer, N., Parmar, N., et al. “Attention Is All You Need.” NeurIPS, 2017. arXiv:1706.03762.

Appendix A. V7 Experimental Record Schema

Field	Example	Purpose
trial_id	v7_00017	Unique intervention identifier
parent_generation	10	Parent checkpoint
parent_checkpoint_sha256	stored hash	Artifact integrity
controller_version	controller_0.4	Policy reproducibility
benchmark_version	bench_0.3	Evaluation provenance
target_capability	science_qa	Failure diagnosis
diagnosis_distribution	knowledge=.55; retrieval=.25; reasoning=.20	Controller state
intervention	targeted_learning	Candidate action
compute_budget	100 steps	Matched resource control
seed	17	Stochastic reproducibility
target_score_before	stored value	Baseline
target_score_after	stored value	Candidate
regression_before	stored value	Retention baseline
regression_after	stored value	Retention outcome
retrieval_sources	5	Access provenance
accepted	true/false	Decision

reason	retention criterion passed	Auditability
--------	----------------------------	--------------

Appendix B. Reproducibility Checklist

- Freeze benchmark and held-out partitions before intervention selection.
- Record source revision, checkpoint hash, architecture, optimizer, seed, hardware, and environment.
- Hash corpus and benchmark artifacts.
- Log acquisition queries, URLs, timestamps, hashes, and filtering decisions.
- Store raw benchmark outputs and scorer versions.
- Keep retrieval, tool routing, and parameter-learning outcomes separate.
- Run regression tests after each accepted candidate.
- Preserve rejected candidates, interrupted runs, and resource failures.
- Maintain an untouched copy of the parent checkpoint.

Appendix C. Controller Pseudocode

INPUT: parent checkpoint M, target benchmark B, regression suite G, resource budget C

1. Evaluate M on B and G.
2. For each failure, estimate cause probabilities over {knowledge, retrieval, reasoning, tool, optimization, forgetting, infrastructure}.
3. Select intervention a using the controller policy.
4. If a requires acquisition, collect versioned public sources with provenance.
5. Construct bounded target data and optional replay data.
6. Clone M → M' and apply a under budget C.
7. Evaluate M' on B_target and G.
8. Compute target gain, regression, routing effects, retrieval quality, and resource use.
9. Accept M' only if the predeclared acceptance rule passes.
10. Append diagnosis, scores, hashes, and decision to the lineage.
11. Continue from the accepted parent state.

Appendix D. Benchmark Templates

The following templates illustrate the intended structure of evaluation items. They are templates rather than test results and should be instantiated independently before the final evaluation set is frozen.

Capability	Template	Scoring
Arithmetic	“Compute A op B.” with varied surface forms	Exact numeric equality
Science	“Explain concept C and state one consequence.”	Fact + explanation rubric
Current QA	“What is currently reported about topic T?”	Evidence-grounded correctness
Reading	Passage P + question Q	Answer supported by P
Coding	Program fragment + expected output	Exact output / execution check
Tool routing	Task description + candidate tool set	Correct routing decision

Appendix E. Claim-Status Guide

Claim type	Status	Permitted wording
Generation-10 architecture / parameter count	Observed	“The archived state reports ...”
3,520 steps / 4.64M corpus characters	Observed	“The experiment log records ...”
Nine accepted changes	Observed	“The lineage contains ...”
V7 targeted learning improves held-out	Hypothesis	“V7 tests whether ...”

performance		
JARVIS internalized retrieved facts	Unproven until controlled transfer is shown	“The system retrieved evidence; parameterized learning was not established.”
Frontier parity / general intelligence	Not claimed	Do not present as an observed result

Appendix F. Archived Generation-10 State

Field	Archived value
Generation	10
Training steps	3,520
Accepted changes	9
Best evaluation loss	0.11632630974054337
Corpus characters	4,643,495
Web pages	40
Memory records	103
Parameters	156,144,084
Device	CUDA
Checkpoint	checkpoints/generation_000010.pt

Appendix G. Artifact Organization

- source/ — versioned implementation and controller code.
- checkpoints/ — immutable generation artifacts with hashes.
- corpus/ — acquired and normalized learning material by version.
- benchmarks/ — development, validation, regression, and final-test sets.
- experiments/ — per-trial configuration, outputs, metrics, and decisions.
- web/ — queries, source metadata, content hashes, and filtering logs.
- figures/ — figures with generation scripts.
- repro/ — environment specifications and restore instructions.

Appendix H. Resource and Recovery Notes

The archived experiments were performed on CUDA with a T4-class accelerator and a notebook environment. GPU memory exhaustion occurred for some larger candidates. Because notebook runtimes are temporary, a production-quality research workflow should keep persistent artifacts outside the runtime filesystem and verify checksums before restoration.

A recovery sequence should be deterministic: obtain the intended release artifact, verify the archive checksum, extract it, verify the checkpoint file size and checksum, restore the parent state, and only then enable further training. The baseline should remain available even after later generations are created.

Author note. This manuscript distinguishes what the current system has demonstrated from what the next experiment is designed to test. That separation is intentional: the purpose of JARVIS is to produce interpretable evidence about learning mechanisms, not to substitute model size or narrative for measurement.

Appendix I. Archived Generation-10 Architecture Configuration

The following configuration is transcribed from the archived Generation-10 state and is included to make the parent checkpoint scientifically reproducible. Values describe the archived architecture; they are not proposed V7 settings.

Parameter	Value	Experimental interpretation
vocab_size	256	Symbol vocabulary used by the archived model
d_model	384	Hidden representation width
n_heads	8	Attention heads
n_layers	12	Transformer depth
experts	7	Number of mixture-of-experts branches
top_k_experts	4	Experts selected per routed token
dropout	0.0	Archived dropout setting
block_size	1,280	Context window used by the archived configuration
parameters	156,144,084	Tracked parameter count
device	CUDA	Accelerator execution path recorded in the archive

The table is intentionally separate from the lineage table. Architecture configuration answers “what was the parent model?” whereas lineage answers “how did the system reach that state?” Both are required for a complete experimental description.

Appendix J. Experimental Budgeting and Statistical Decision Rules

A continual-learning controller operates under a finite compute budget. The experimental design should therefore allocate resources before observing candidate outcomes. A practical V7 block can specify a fixed number of training steps per candidate, a maximum number of candidate interventions, a fixed retrieval budget, and a maximum wall-clock duration. These quantities should be recorded in the experiment manifest.

Quantity	Recommended control	Reason
Training steps per candidate	Fixed in advance	Prevents hidden compute advantage
Random seeds	Multiple when feasible	Separates robust gains from seed sensitivity
Retrieval calls	Bounded and logged	Controls information-access budget
Candidate count	Predeclared per block	Avoids unbounded search on validation data
Validation set	Frozen within block	Limits adaptive overfitting
Final test set	Untouched until final evaluation	Protects generalization estimate
Regression tolerance τ	Predeclared	Prevents post hoc acceptance
Minimum target gain δ	Predeclared	Defines practically meaningful improvement

The statistical target is not to maximize the number of hypothesis tests. It is to obtain interpretable evidence from a limited number of interventions. For paired benchmark outcomes, effect estimates should be reported with confidence intervals, and multiple candidate comparisons should be interpreted in light of the selection procedure. The controller should not repeatedly mine a validation set until a favorable result appears.

When the number of benchmark items is small, exact or permutation-based inference is preferable to relying on asymptotic approximations. When repeated seeds are available, mean improvement should be reported alongside dispersion. A particularly informative quantity is the fraction of candidate interventions that pass the acceptance rule, since a controller that produces rare but large gains differs scientifically from one that produces small but highly reproducible gains.

Appendix K. Data Provenance and Contamination Controls

The V7 learning system should maintain a provenance graph linking information sources to training examples, candidate checkpoints, and evaluation outputs. Every acquired document receives a stable record containing its source

URL, acquisition timestamp, query, source domain, content hash, and filtering decision. Training examples derived from the document carry the source identifier forward.

Contamination checks should operate at several levels. Exact overlap detection can identify copied evaluation strings. Near-duplicate detection can identify templated or lightly paraphrased leakage. Source-level separation can ensure that a final-test question was not built from the same source page used to construct a training example. The benchmark manifest should record the rule used to create each item so that a later audit can determine whether it was eligible for training exposure.

Control	Implementation	Failure it prevents
Exact-overlap check	Hash normalized strings and search training corpus	Direct memorization of test text
Near-duplicate check	Character/word n-gram similarity or embedding review	Template leakage
Source separation	Keep final-test sources outside acquisition set	Source memorization masquerading as transfer
Version locking	Freeze benchmark/data revisions during a block	Moving-target evaluation
Output archival	Store raw predictions	Silent scorer changes
Provenance propagation	Link training examples to source IDs	Untraceable learning events

The provenance graph also supports negative-result analysis. If a targeted intervention fails, the investigator can inspect which sources were used, whether the training examples were diverse, and whether the candidate actually received the intended information. This prevents a failed result from being misinterpreted as a failure of the learning hypothesis when the acquisition stage itself malfunctioned.